Some terms you'll run across are *relations*, *tuples*, *bloat*, and the *optimizer*.

A *relation* refers to a table. *Tuples* are versions of rows in tables. *Bloat* has to do with inefficient data layout, where data is stored in a fragmented way and this fragmentation can add latency to operations. These very brief definitions are a starting point for exposure to these terms.

Some basic information on PostgreSQL data storage internals is useful. There's a lot more that we won't discuss.

In PostgreSQL, data is laid out in *pages*. Data cannot be split across pages. Data is stored efficiently when there are minimal gaps and data that is accessed resides within a single page. Data that's in a single page requires fewer disk operations compared with accessing data that is split across multiple pages.

Another concept on data layout is *correlation*. Correlation refers to the relationship between how data resides on disk and how the data is ordered in a query. High correlation means they're similar and low correlation means they are dissimilar.

PostgreSQL works whether data is stored efficiently or not. Data may not fit in a single page, may be split across pages, and there may be a low correlation for a query. You will explore these concepts.

Next let's look at some SQL terminology.

Learning SQL Terminology

SQL has loads of terms as well, and much of it is not specific to PostgreSQL.

SQL is a *declarative* language. As a programmer, you *declare* to PostgreSQL what results you want and PostgreSQL will figure out the best plan to get them.

You specify the columns from the tables you want, and PostgreSQL determines an optimized retrieval algorithm using knowledge of the data layout, database objects, available hardware resources, and more.

PostgreSQL creates *query execution plans* on the fly to fulfill what you've asked for. Multiple plans are pitted against one another. The plans are formed using estimates about data in tables, so the plans aren't always flawless but are generally pretty good. As the programmer, you can see what PostgreSQL is planning and influence it. You'll learn more about that later on.

This plan evaluation process is conducted nearly instantly by the query execution planner, which is also referred to as the *optimizer*.